El estándar Screenplay aplicado a mis 4 proyectos

> unas 1000 palabras por proyecto · repo `greenroom-qa` · verificado contra el código real

El estándar (vale para los 4)

Screenplay es un patrón de automatización donde las pruebas se escriben como una obra: un **Actor** (quién) con una **Ability** (con qué puede actuar, aquí `BrowseTheWeb` = la página de Playwright) ejecuta **Tasks** (acciones de negocio reutilizables: *abrir, buscar, navegar*) y formula **Questions** (lecturas verificables: *¿se ve? ¿cuántos hay? ¿qué dice?*). Los selectores viven en **Page Elements**, nunca en la prueba. Reglas del estándar: el spec solo orquesta intención de negocio; cada Task/Question tiene un nombre que se entiende sin leer su código; lo que se repite en 2 pruebas se convierte en Task; lo que se afirma se convierte en Question; los datos vivos se validan por estructura, nunca por valor exacto. En `greenroom-qa` el núcleo vive en `src/core/` (`actor.ts` , `tasks.ts` , `questions.ts`) y cada suite lo reutiliza. Dicho esto, cada proyecto merece su propio elenco.

---

1. Terramind — mapa 3D ambiental + copiloto RAG (~1000 palabras)

Terramind es la app más compleja de las cuatro: mapa 3D con decenas de estaciones, capas conmutables (aire, lluvia, agua, vegetación, comunas), dashboard de 11 pestañas y un copiloto RAG que responde con normativa. Aquí Screenplay no es un lujo, es la única forma de que la suite no se pudra: los flujos cruzan mapa → panel → chat, y con POM puro cada Page Object terminaría conociendo a los demás.

Actor y Ability. El Actor se llama `Santiago, el Analista Ambiental`: abre la app buscando el estado del aire del Valle de Aburrá. Su Ability es `BrowseTheWeb` (la `Page` de Playwright con Chromium). En el código es `Actor.named('Santiago', page)`. Si mañana se prueba el modo multiusuario (ciudadano vs administrador), no se reescribe nada: se crean dos actores con la misma Ability y el resto del elenco se reutiliza. Esa es la primera lección defendible en entrevista: el *quién* está separado del *cómo*.

Tasks propias. Las genéricas (`OpenApp`) abren, pero Terramind necesita tareas de dominio: `ConsultarAQIdeComuna("Belén")` (localizar el marcador, clicarlo, leer el panel), `AlternarCapa("Clima y Radar")` (clic en la pestaña lateral y espera a que el canvas re-renderice), `AbrirDashboard` y `PreguntarAlCopiloto("¿Cuál es la calidad del aire hoy?")` (abrir el panel RAG, escribir en `textarea.chat-input`, enviar). Cada una encapsula sus esperas: el mapa es WebGL y tarda segundos en pintar estaciones, así que la espera vive dentro de la Task, nunca como `sleep` en el spec. Nuestra suite actual ya hace esto en pequeño: el test del copiloto clica el botón RAG antes de buscar la caja de texto, en vez de asumir que el chat está visible.

Questions propias. `ValorAQIComuna` (lee "PM2.5 17.4 · AQI 60" del panel y devuelve el número, sin afirmar un valor fijo porque SIATA es en vivo), `EstacionesVisibles` (conteo de marcadores > 0), `PanelPrincipalVisible`, y `RespuestaCopiloto` (¿apareció burbuja de respuesta, aviso de demo o indicador de carga?). La regla de oro aquí: el AQI de hoy será distinto mañana; la Question responde el dato y el spec solo afirma estructura ("hay un número", "hay marcadores"). Quien afirme `AQI == 60` en un test contra datos vivos no entendió caja negra.

Page Elements. El mapa es canvas: no hay nodos DOM por estación, así que los Elements son regiones (`canvas` del mapa, panel `.aqi-card`, `textarea.chat-input`, botón `Enviar`, sidebar de capas). Documentar *por qué* se usa cada localizador es parte del estándar: `getByRole` para botones, CSS para el canvas, placeholder para el chat ("Pregunta a los documentos...").

Ejemplo real del repo. Nuestro test de mapa: `santiago.attemptsTo(OpenApp.now())`, luego pregunta `PageTitle`, `IsVisible('canvas')`, y falla si hay `pageerror` críticos. Tres líneas que cualquier persona lee: abrir, preguntar título, preguntar mapa, cero errores. Eso es Screenplay.

Por qué Screenplay aquí y no POM. El flujo "consultar comuna → abrir RAG → preguntar por esa comuna" encadena tres páginas lógicas; en POM serían tres Page Objects llamándose entre sí. En Screenplay es una Task compuesta `InvestigarComuna` que reusa las tres Tasks simples. Cuando cambie el diseño, se toca una Task, no doce tests. Y la prueba pendiente del copiloto (skip honesto por inestabilidad headless) demuestra criterio profesional: un skip documentado vale más que un verde maquillado.

Ampliación: el resto del elenco. El mapa tiene más capas que el aire: `VerRadarLluvia` (activa "Clima y Radar" y verifica que el canvas pinta precipitación), `VerNivelesAgua`, `VerVegetacion` y `CambiarVista2D_3D` (el botón 2D/3D superior derecho). Cada capa es una Task de una línea en el spec, y su Question gemela (`RadarVisible`, `NivelAguaVisible`) afirma que el layer existe sin juzgar los milímetros de lluvia, que son en vivo. El dashboard de 11 pestañas se cubre con `AbrirPestana(nombre)` parametrizada: un solo método, once casos vía tabla de ejemplos (Estaciones, Agua, etc.), cada uno preguntando `ContenidoPestanaVisible`. Así se escala sin multiplicar código: añadir la pestaña 12 es añadir una fila, no un test.

Ability con esteroides. `BrowseTheWeb` en Terramind incluye escuchar errores: nuestra suite registra `pageerror` y falla si hay críticos (filtrando ruido como `favicon` o `net::`). Eso, envuelto como Question `SinErroresCriticos`, convierte la consola del navegador en un oráculo de calidad: un despliegue roto en Vercel se detecta aunque la pantalla "se vea bien". Para datos, las Tasks aceptan parámetros de una tabla de comunas (Belén, Copacabana, Envigado, Itagüí, Sabaneta, La Estrella): el mismo escenario corre 6 veces con 6 datos. Si una comuna no pinta, el reporte dice cuál, no solo "falló el mapa".

Mantenimiento y entrevista. El caso RAG enseña la disciplina de lo inestable: tres intentos (fill directo, teclado, `--disable-gpu`), evidencia en cada uno (screenshots y traces), y al final skip documentado con causa raíz probable (crash WebGL en headless). En entrevista se cuenta así: "verifiqué manual que el chat existe, automaticé lo automatizable y declaré lo pendiente". También se defiende el presupuesto: priorizar mapa+panel (lo que todos ven) sobre el copiloto (lo que pocos usan) es gestión de riesgo, no pereza. Y cuando el backend exponga API de estaciones, las mismas Questions servirán para contrastar UI contra API: `ValorAQIComuna` se compara con `GET /estaciones`, cerrando el círculo caja negra/caja gris.

Cierre: cómo se defiende. "Elegí Screenplay porque el flujo cruza tres superficies con estado compartido (comuna seleccionada) y las Tasks componen; las esperas WebGL viven en las

Tasks; los datos vivos se afirman por estructura; lo inestable se declara con evidencia, no se maquilla; y la matriz de comunas × capas corre el mismo spec con N datos". Eso, dicho mirando a los ojos, es el estándar aplicado, no recitado.

---

2. Pulso GitHub — ranking estático de repositorios (~1000 palabras)

Pulso es lo opuesto a Terramind: un sitio estático que publica el ranking de repos en español, con tabla, buscador y filtros por tema. Aquí la lección es de honestidad técnica: Screenplay *puede* usarse, pero hay que saber decir *cuándo no paga*.

Actor y Ability. `Santiago, el Lector`: entra a ver qué repos lideran esta semana. Ability `BrowseTheWeb` igual que siempre. Como no hay roles ni login, un solo actor basta; crear más sería teatro innecesario, y decir eso en entrevista suma puntos: el patrón se aplica con criterio, no por moda.

Tasks propias. `AbrirRanking` (navegar a la URL completa del `.html`, porque el root de Pages da 404: defecto real documentado), `BuscarRepo("query")` (llenar el buscador y Enter), `CambiarCategoria("python")` (clic en filtro y espera a re-render). Son tan simples que cada una cabe en cinco líneas; el valor no está en la complejidad sino en el vocabulario: el spec dice *buscar* y *cambiar*, no *fill* y *click*.

Questions propias. `CantidadRepos` (conteo de filas/tarjetas > 0), `ResultadosFiltrados` (el conteo tras buscar es menor o igual que antes) y `VistaActualizada` (tras cambiar categoría sigue habiendo contenido). Los datos cambian cada semana con la actualización automática, así que jamás se afirma un nombre de repo ni un número exacto: estructura y presencia, siempre. Nuestra suite hace exactamente eso y por eso es estable.

Page Elements. Tabla (`table tbody tr`), tarjetas (`.repo` , `article`), enlaces a github.com, buscador (`searchbox` o placeholder) y filtros (`tab/button/combobox` por nombre de tema). Un detalle profesional que ya aplicamos: el buscador puede no existir según la versión, así que la Question lo tolera con `skip` justificado en vez de reventar.

Ejemplo real del repo. `const rows = await santiago.asks(CountOf.elements('table tbody tr, article, li')); expect(rows).toBeGreaterThan(0);` — el test declara *cuántos hay* y afirma *más de cero*. Si el ranking crece de 500 a 600 repos, el test sigue verde. Eso es diseñar contra la volatilidad del SUT.

Por qué (casi) POM aquí. Con un smoke de una página y tres interacciones, POM es menos código y suficiente. Screenplay se justifica en Pulso solo si crece: comparar dos temas lado a lado, paginar el top 500, o validar el deep-link `?mod=tre` serían Tasks compuestas (`CompararTemas` , `RecorrerRanking`) donde el patrón brillaría. La respuesta de entrevista: "Screenplay donde hay

flujo multi-paso reutilizable; POM donde hay una página y tres clics". Pulso hoy es lo segundo, y lo decimos sin vergüenza: elegir el patrón barato cuando basta *es* criterio senior.

Ampliación: casos que sí merecen Tasks. El deep-link `02-midnight-console.html?mod=tre` es una entrada directa a la consola de medianoche: merece la Task `AbrirConsolaDirecta(url)` y la Question `ModuloActivo("tre")`, porque valida algo que el flujo feliz nunca toca (bookmarks compartidos, links de redes). El bug real que encontramos (el root de Pages devuelve 404) se convierte en caso negativo permanente: `AbrirRaiz` debe responder 404 o landing, nunca pantalla en blanco; documentado como defecto con severidad media y recomendación (crear `index.html`). El buscador merece su matriz de equivalencia: texto normal (filtra), texto inexistente (`zzzz-123`, reduce a cero o muestra vacío), vacío (restaura todo) y caracteres especiales (no rompe). Cada clase es un caso; la Task es la misma. Los filtros por tema se parametrizan en tabla (python, javascript, todos...): si mañana agregan Rust, se agrega una fila.

Rendimiento y honestidad de datos. Un smoke puede incluir una Question de tiempo (`CargaEnMenosDe(segundos)`) midiendo `domcontentloaded` sin afirmar milisegundos exactos: en Pages gratis hay varianza, así que el umbral es generoso y documentado (p95 < 10 s, por ejemplo). Y la honestidad de datos: el ranking se regenera semanalmente vía Actions, así que el README de la suite declara "los valores cambian; se valida estructura". Esa frase, escrita antes de que pregunten, es seniority visible.

Cuándo graduar Pulso a Screenplay completo. Tres señales: (1) aparece login o personalización (nace el segundo actor), (2) hay un flujo de 3+ pasos que se repite (comparar, filtrar, exportar), (3) dos suites necesitan la misma Task. Mientras tanto, el `src/core` compartido (OpenApp, SearchFor, CountOf) ya deja la puerta abierta sin sobre-ingeniería: Pulso usa el elenco común y solo crearía Tasks propias el día que las necesite. Decirlo así en entrevista muestra que dominas el patrón *y* sabes cuándo no usarlo, que es justo lo que evalúa un líder técnico.

Cierre: calendario y accesibilidad. Como el ranking se regenera semanalmente, la suite corre en cron nocturno: si un lunes el deploy amanece roto, el reporte lo dice antes que los usuarios. Suma dos Questions baratas de accesibilidad (`ContrasteSuficiente` queda para axe-core; hoy basta `NavegablePorTeclado`: el buscador recibe foco con Tab y responde a Enter, porque el público de Pulso incluye lectores de pantalla). Y el mantenimiento es una línea al mes: revisar que los selectores sigan existiendo tras cada rediseño. Poco código, mucha señal: el estándar también es saber cuánto automatizar.

---

3. Query Lands (LAB.CORE) — retos SQL gamificados (~1000 palabras)

Query Lands es una plataforma Next.js donde un estudiante avanza por islas completando retos SQL con editor en el navegador, tutor IA y XP. Es el caso perfecto para Screenplay de manual: flujo de varios pasos con estado (reto abierto → consulta escrita → resultado → XP).

Actor y Ability. `Santiago, el Estudiante`: entra a practicar SQL para entrevistas. Ability `BrowseTheWeb`. A futuro, un segundo actor `Tutor` (rol que revisa progreso) mostraría la fuerza del patrón: dos actores, mismo escenario, distintas Tasks. Por ahora uno basta, pero el diseño ya lo permite.

Tasks propias. `AbrirReto` (clic en el botón CONTINUAR —ojo, no en el nav "Desafíos", que lleva al listado: distinción aprendida a golpes y documentada), `EscribirConsulta("SELECT ...")` (llenar el editor Monaco/textarea), `EjecutarConsulta` (clic en ejecutar y espera a la zona de resultado). La lección técnica central: es una SPA, así que `waitForLoadState('domcontentloaded')` no espera nada tras navegar; la espera real es la aparición del editor. Esa espera vive dentro de la Task (`expect(...).toBeVisible({ timeout })`), que es exactamente para lo que existen las Tasks: esconder el *cómo esperar*.

Questions propias. `RetosVisibles` (hay botones/enlaces en home), `EditorVisible` (existe textarea, Monaco, `code` o `pre` tras abrir el reto), `ResultadoConsulta` (aparece texto de resultado, corrección o error de validación) y a futuro `XPActualizado` (la meta diaria avanzó). Ninguna afirma el texto exacto del enunciado ni el resultado de la query: el contenido lo genera el tutor IA y cambia; se afirma *que hay* editor, *que hay* veredicito. Nuestra suite pasó de fallar a 3/3 verde justo cuando dejamos de asumir navegación instantánea.

Page Elements. Botón CONTINUAR (localizado por texto, no por posición: si cambia de lugar, sigue funcionando), editor (`textarea` , `[role="textbox"]` , `.monaco-editor`), botón ejecutar (por nombre: Ejecutar/Correr/Run/Validar) y región de resultado (por texto: resultado/correcto/error/fila). Nótese el principio: localizar por *intención visible* (roles y textos), nunca por clases CSS generadas ni por índices. Si el equipo rediseña, los roles sobreviven; los `.css-1a2b3c` , no.

Ejemplo real del repo. El test de abrir reto: `OpenApp` → clic CONTINUAR → `expect(editor).toBeVisible({ timeout: 20000 })` . Cuando falló la primera vez, el screenshot mostró que seguíamos en home: el clic había ido al nav equivocado. El diagnóstico quedó

registrado y el fix fue un localizador (CONTINUAR exacto), no un `sleep`. Esa historia, contada en entrevista, vale más que diez verdes fáciles.

Por qué Screenplay aquí. El flujo completo "abrir → escribir → ejecutar → verificar XP" es una Task compuesta natural (`CompletarReto`) hecha de tres Tasks simples, reutilizable para los 16 retos SQL y los 3 de JS parametrizando el enunciado. En POM serían métodos encadenados entre Page Objects con estado compartido a escondidas; en Screenplay la composición es explícita y cada pieza se prueba sola. Además el انتظار es asíncrono por naturaleza (ejecución SQLite/WASM + IA): concentrarlo en Tasks/Questions es aplicar el patrón donde más duele.

Ampliación: el resto del viaje. La home tiene más que CONTINUAR: `IrARepasar` (repaso espaciado), `IrADesafios` (listado), `IrACursos` , `IrAPerfil` y `EmpezarOnboarding` (la tarjeta "¿Primera vez?"). Cada destino es una Task de navegación y su Question de llegada (`EstoyEnRepaso` , `PerfilVisible`), así el mapa de navegación queda cubierto sin repetir localizadores: el nav lateral vive en un solo Page Element. El botón CONTINUAR merece mención especial como estándar: se localiza por texto visible, se espera con reintento (la home carga el reto por API) y su ausencia produce `skip` , no `fail` , porque un home sin reto del día es contenido, no bug.

El editor como actor secundario. Escribir en Monaco no es un `fill` cualquiera: requiere foco, tecleo y a veces `Ctrl+Enter` para ejecutar. La Task `EscribirConsulta` encapsula ese ritual (clic, `pressSequentially` con delay humano, verificación de que el texto quedó), y `EjecutarConsulta` acepta tanto el botón como el atajo de teclado según lo que exista. Los mensajes de validación se catalogan como Questions (`ResultadoCorrecto` , `ErrorSintaxis` , `PistaMostrada`): si el tutor cambia su redacción, se actualiza la Question, no los 19 retos. Y el XP: `MetaDiariaAvanzó` compara el "0/30 XP" antes y después, afirmando progreso sin afirmar cantidades exactas (el puntaje por reto puede re-balancearse).

Datos y entorno. Los 3 retos JS comparten Tasks con los 16 SQL cambiando solo el lenguaje del editor: parametrización, no duplicación. Si algún día hay login, nace el segundo actor con sus credenciales por entorno, igual que en EDU.CORE. Y la lección de flakiness queda escrita: dos corridas idénticas dieron distinto porque la home es asíncrona; el fix fue espera con reintento en la Task, jamás `sleep` . Esa historia, con fechas y capturas, es la evidencia que un reclutador no puede discutir.

Cierre: persistencia y móvil. El progreso del estudiante vive en el navegador (localStorage) o en cuenta: la Question `ProgresoPersistente` recarga la página y verifica que el XP sigue ahí, cubriendo el bug más odiado de las apps gamificadas (perder el avance). En móvil, los retos se abren igual pero el editor se achica: una Task `RotarVista` o un proyecto Playwright con

viewport de teléfono extiende la cobertura sin duplicar specs (mismo elenco, distinto dispositivo). Y cuando el catálogo crezca a 30 retos, la tabla de datos crece sola: el estándar escala agregando filas, no archivos.

---

4. EDU.CORE — gestión estudiantil con login por roles (~1000 palabras)

EDU.CORE (repo `plataforma-estudiantil`) es un sistema con backend Spring Boot y frontend comercial: landing, login con pestañas Iniciar Sesión/Registrarse, registro con rol (Estudiante/Docente/Familiar) y panel por rol. Es el proyecto donde Screenplay demuestra su mejor truco: **actores parametrizados y datos por entorno**.

Actor y Ability. Tres actores, una Ability: `Santiago, el Estudiante`, `Paola, la Docente` y `Luis, el Acudiente`, todos con `BrowseTheWeb`. Cada uno entra con credenciales distintas desde variables de entorno (`EDUCORE_USER`, `EDUCORE_PASS`), nunca quemadas en el repo. Esta es la respuesta a "¿para qué sirven varios actores?": el mismo escenario de login se ejecuta tres veces cambiando solo el actor y sus datos, y cada rol debe aterrizar en su panel. En POM eso serían tres métodos casi idénticos; aquí es un parámetro.

Tasks propias. `IniciarSesion(usuario, clave)` (clic en pestaña "Iniciar Sesion", llenar `nombre@institucion.edu.co` + `Tu contrasena`, clic en "Entrar a mi portal"), `RegistrarCuenta(nombre, correo, clave, rol)` (el formulario con validaciones: nombre ≥ 2 caracteres, clave ≥ 6, selector de rol) y `NavegarPanel(seccion)` (Asistencia, Biblioteca, Cursos, Evaluaciones). Fíjate que los labels y placeholders son literales del componente `CommercialLoginView.jsx`: la Task cita su fuente. Y el caso inválido es determinista sin backend: clave de 3 caracteres dispara la validación local ("mínimo 6 caracteres"), así que el test es rápido y estable aunque Render esté dormido.

Questions propias. `MensajeError` (el bloque titulado "No se pudo continuar" con su detalle), `PanelVisible` (texto de bienvenida/dashboard tras login válido), `RolActivo` ("Tu rol se activa automáticamente al entrar") y `FormularioRegistro` (existen nombre/correo/clave/rol). La regla de datos sensibles: los tests solo usan cuentas de prueba por entorno; si `EDUCORE_USER` no existe, el test de login válido hace `skip` declarado, no `fail` injusto. Un login que falla por falta de credenciales no es un defecto del sistema: es del entorno, y el reporte debe decirlo así.

Page Elements. Pestaña "Iniciar Sesion" (botón por nombre), inputs por placeholder exacto, submit "Entrar a mi portal", bloque `.cli-diagnostic.error`, selector de rol con opciones ESTUDIANTE/DOCENTE/FAMILIAR. Todo accesible por rol o texto: si el equipo cambia los estilos, nada se rompe. Nuestra suite ya sobrevivió a un rediseño completo precisamente por eso (los selectores viejos genéricos se reemplazaron por los literales nuevos sin tocar la lógica).

Ejemplo real del repo. Test inválido: pestaña → `fill('invalido@test.com')` → `fill('123')` → clic Entrar → `expect("No se pudo continuar")`. Cinco líneas, cero ambigüedad, cero backend. Y el válido queda en skip hasta que Render publique la nueva UI: documentado, con commit `968690e` como referencia. Eso es gestión de dependencias de entorno, tema favorito de entrevistadores.

Por qué Screenplay aquí. Login × 3 roles × 2 modos (login/registro) × N módulos del panel es una matriz que en POM explota en métodos duplicados; en Screenplay es `IniciarSesion` × actor × datos. Además el registro tiene reglas de negocio (longitudes, rol) que se expresan como Tasks con validación, y los mensajes de error como Questions: el día que el backend cambie un mensaje, se actualiza una Question. Cierra el círculo con la oferta: esto es diseñar automatización mantenible con Page Objects donde tocan (los Elements) y Screenplay donde brilla (los flujos por rol).

Ampliación: la matriz completa. Escrita como tabla, la cobertura canta sola: filas = roles (Estudiante, Docente, Familiar), columnas = acciones (login válido, login inválido, registro válido, registro inválido, navegar a cada módulo, cerrar sesión). Cada celda es el mismo spec con distinto actor y datos; las Tasks no cambian. `CerrarSesion` merece ser Task de primera clase (salir y verificar que el login reaparece), porque las sesiones pegadas son la primera causa de falsos verdes en suites con login. Los módulos del panel (Asistencia, Biblioteca, Cursos, Evaluaciones, Dashboard) se cubren con `NavegarPanel(nombre)` + `ModuloVisible(nombre)`: si el rol Familiar no debe ver Evaluaciones, eso es un caso negativo explícito (`ModuloAusenteParaRol`), no un olvido.

Registro y sus trampas. El formulario de registro es una mina de valores límite: nombre de 1 carácter (falla), clave de 5 (falla) y de 6 (pasa), correo sin @ (falla por `type=email` del navegador antes de llegar al backend). Cada límite es un caso con su mensaje esperado, y todos viven como datos del mismo test. El botón demo (`onDemo`) merece su propia Task (`EntrarComoDemo`) porque es el camino que los revisores manuales usan a diario: automatizarlo les ahorra minutos cada vez. Y el selector de rol (ESTUDIANTE/DOCENTE/FAMILIAR) se manipula por valor visible, de modo que si agregan "ADMIN", el cambio es una fila más.

Backend y futuro API. El panel consume API Spring Boot; cuando Karate entre al repo, las Questions de UI se contrastan con `GET` (el panel muestra lo que el API devuelve) y el registro se prueba también por `POST` con payloads válidos/inválidos: la misma matriz, dos capas. Las credenciales de prueba saldrán del `DataLoader` del backend documentado en el repo, cerrando el lazo entre quien siembra datos y quien los verifica. Esa es la historia completa para la oferta: UI con Screenplay hoy, APIs con Karate mañana, mismo vocabulario de actores y datos.

Cierre: sesiones y roles. La sesión vive en cookies/token: la Task `RecordarSesion` recarga tras login y verifica que el panel sigue ahí, y `SesionExpirada` (si el backend expira) espera el redirect al login en vez de colgarse. Cada rol con su panel es un contrato: si Docente ve Evaluaciones y Estudiante no, ambas afirmaciones conviven en la matriz sin contradecirse porque el actor es distinto. Y el resumen para entrevista: "parametrizo actor × datos sobre una matriz rol-acción, las validaciones de negocio viven en Tasks, los mensajes en Questions, los secretos en entorno y lo no desplegado en skip declarado". Matriz, no maraña.